

Problem Statement & Objectives

AUTOMATED KUBERNETES NETWORK SELF-HEALING

THE PROBLEM

✖ Silent Failures

DNS outages, misconfigured policies, or CNI crashes occur without pod termination, bypassing standard K8s health checks.

✖ Manual Bottleneck

Traditional monitoring relies on human operators, leading to high MTTR and prolonged service downtime.

THE OBJECTIVE

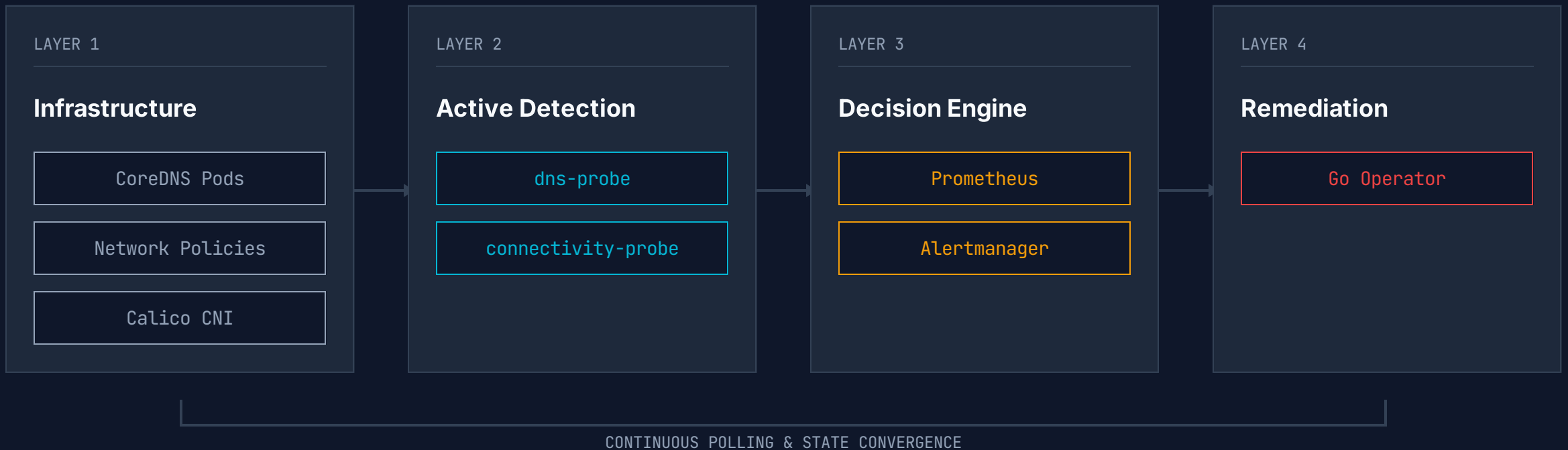
Design and implement a closed-loop, zero-touch self-healing system for Kubernetes network infrastructure.

KEY GOALS

- 01 Active Detection:** Continuously verify DNS resolution & cross-namespace connectivity.
- 02 Automated Remediation:** Programmatically resolve faults without human intervention.
- 03 Real-time Observability:** Visualize the failure-and-recovery lifecycle in real-time.

4-Layer Self-Healing Architecture

SYSTEM DESIGN & MECHANISM OVERVIEW



What We Built, Fixed & Evaluated

EXTENSIONS, CRITICAL ISSUES & CLOSED-LOOP RESULTS

KEY IMPLEMENTATIONS & EXTENSIONS

Custom Kubernetes Operator

Go-based controller interacting directly with the Kubernetes API to execute automated remediation logic.

Active Probing System

Python exporters, dns-probe and connectivity-probe, continuously validate cluster network health.

Chaos Engineering Framework

Fault-injection pipeline simulates CNI crashes, blocking NetworkPolicies, and DNS death.

Observability Pipeline

Prometheus, Alertmanager, and Grafana tuned for network self-healing visualization.

CRITICAL ISSUES FIXED

Network Policy Additivity

Operator strips conflicting baseline "allow" rules before applying targeted fixes, solving additive policy stacking.

Metric Tracking Accuracy

Using `kill 1` inside containers simulates true runtime crashes for accurate Kube-State-Metrics tracking.

EVALUATION & RESULTS

- 01 DROP
Fault injection
- 02 ALERT
Prometheus / AM
- 03 HEAL
Go Operator
- 04 RECOVER
Healthy state

✓ CLOSED-LOOP SUCCESS

Injected faults are detected, remediated, and returned to green without human intervention.

Performance, Scalability & Reliability

SELF-HEALING SLA & SYSTEM ROBUSTNESS

PERFORMANCE (SELF-HEALING SLA)

~60s

Detection to Remediation Loop

Probe Interval	15s
Prometheus "For" State	15s
Alertmanager Grouping	10s
Operator Execution	10s
State Convergence	10s

RELIABILITY & SCALABILITY

IDEMPOTENCY

Standard client-go libraries ensure operations are safe to run multiple times without corruption.

RBAC SECURITY

Strict least-privilege scoping via dedicated ClusterRoles and ServiceAccounts.

STATELESSNESS

Go-webhook receiver holds no local state, allowing horizontal scaling across large clusters.

METRIC EFFICIENCY

Lightweight probes conform to standard K8s formats, preventing Prometheus overhead.

Technical Challenges & Learnings

INSIGHTS FROM BUILDING RESILIENT INFRASTRUCTURE



Monitoring "Chicken-or-Egg"

DNS failure broke internal K8s resolution, causing Grafana to lose connection to Prometheus.

LEARNING

Decouple monitoring from the infrastructure it monitors using IP-based routing.



Event-Driven Concurrency

Alertmanager fired multiple webhooks for one incident before the cluster fully healed.

LEARNING

Implement state-aware logic in the Operator to check API state before acting.



Simulating True Outages

Standard pod deletions reset restart counters, masking the true nature of runtime crashes.

LEARNING

Use "kill 1" inside containers to simulate crashes that Kube-State-Metrics can track.